EVALUACIÓN FINAL— DESAFÍO FINAL

DataSalud Perú — Base de Datos Avanzadas y Big Data (CIIN1021P)

Dato	Detalle
Grupo	DataSalud Perú — DIRESA La Libertad
Integrantes	Yef Puntillo Calsin Kenneth Zaidd Machacuay Maldonado Jairo Joao Baltodano Avelino
Docente	Jorge Luis Muñoz Diaz / Nilton Cesar Mercado Chavez

1. Definición del reto, alcance y supuestos

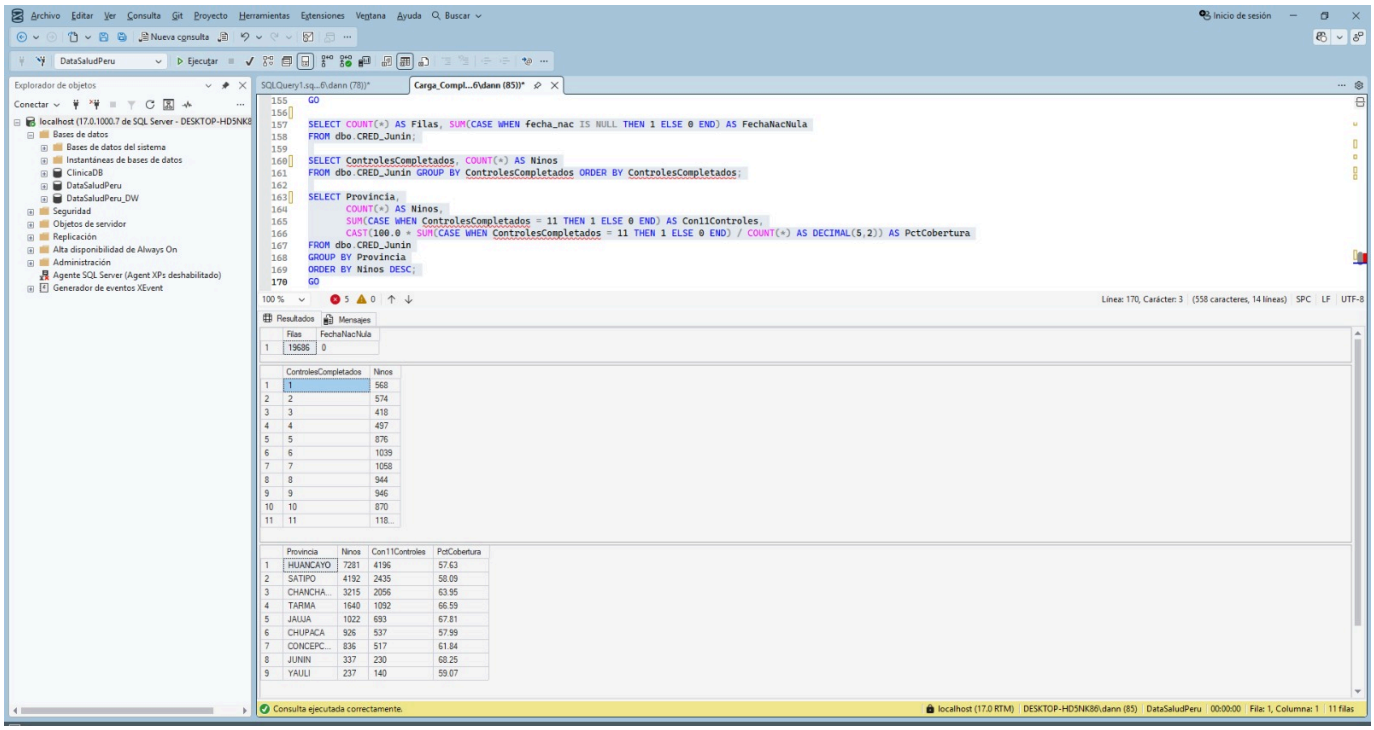
1.1 Datasets seleccionados

Dataset	Fuente / URL	Fecha descarga	N.º filas	N.º columnas
Listado de Pacientes Nuevos (INEN)	INEN — datosabiertos.gob.pe/dataset/listado-de-pacientes-nuevos-enero-2022-julio-2026-instituto-nacional-de-enfermedades	20/09/2026	78,975	9
Niñas y Niños de 364 días con 11 Controles CRED (DIRESA Junín)	DIRESA Junín — datosabiertos.gob.pe/dataset/ni%C3%B1as-y-ni%C3%B1os-de-364-d%C3%ADas-de-edad-11-meses-29-d%C3%ADas-que-cuentan-con-once-11-controles-cred-en	20/09/2026	19,686	52

1.2 Problemas de calidad detectados

N.º	Problema	Dataset / columna	Registros afectados	% del total	Tratamiento
1	Pérdida del cero inicial en código Ubigeo	INEN / UBIGEO_LUGAR_RESIDENCIA	16,757	21.2%	Normalizar con LPAD/relleno de ceros a la izquierda antes de cualquier cruce geográfico
2	El diccionario de datos no coincide con los valores reales (sexo, seguro, flags codificados vs texto)	CRED Junín / sexo, seguro, flag_cnv, num_cred_mes9-11	19,686 (100%)	100%	Validar cada campo contra la data real antes del ETL; documentar la codificación real, no la del diccionario
3	Identificador de paciente/niño no confiable (UUID duplicado / num_doc anonimizado a un solo valor)	INEN / UUID · CRED / num_doc	2 · 19,686	0.003% · 100%	Usar llave sustituta (HISTORIA en INEN; combinación fecha_nac+ubigeo+establecimiento en CRED)
4	Fechas en formato de texto no estandarizado	CRED Junín / fecha_nac, fecha_final, fecha_cred_mes1-11	19,686 (100%)	100%	Convertir a tipo DATE especificando formato d/m/aaaa explícito en la carga a SQL Server

Evidencia de Carga y Resultados CRED Junín (RF1):



Análisis de los resultados de carga:

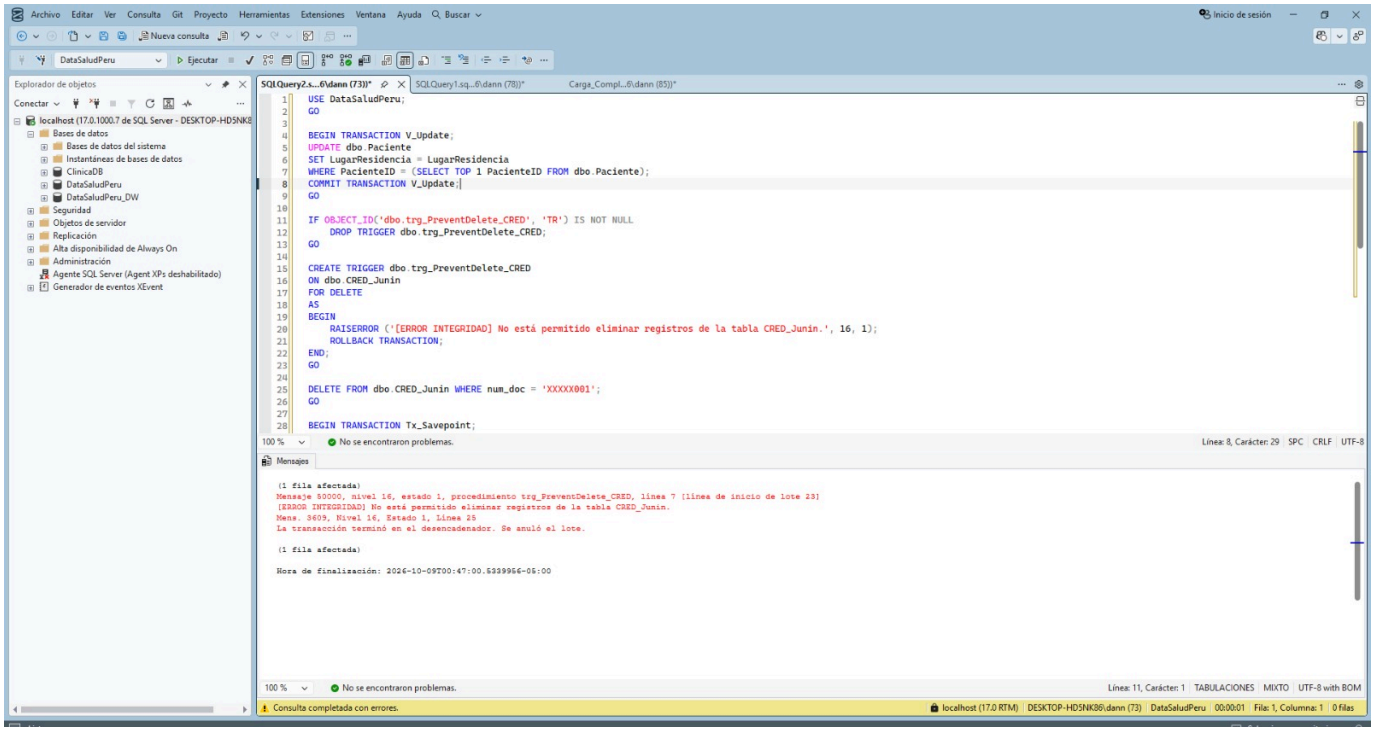
- **Verificación de Ingesta:** Se confirmó la carga limpia de **19,686 registros** en la tabla `dbo.CRED_Junin` con **0 fechas de nacimiento nulas**.
- **Distribución de Controles (Hallazgo de Calidad):** Ningún registro presenta 0 controles; la distribución inicia en 1. Esto confirma que el dataset solo incluye niños registrados con al menos un control. Además, aunque el nombre del archivo sugiere "con once controles", solo el **60.4% (11,896 niños)** cuenta con la serie completa de 11 controles.
- **Cobertura Geográfica:** Se evaluaron las 9 provincias de Junín. La provincia de **Huancayo** concentra la mayor cantidad de niños registrados (7,281) y presenta una cobertura de solo **57.63%**, posicionándose como la provincia prioritaria para la toma de decisiones de la DIRESA Junín.

1.3 Requerimientos funcionales

ID	Requerimiento	Criterio de éxito verificable
RF1	El sistema debe calcular la tasa de cobertura de controles CRED por distrito/provincia	Consulta SQL devuelve el % de niños con 11/11 controles CRED agrupado por provincia, con el UBIGEO ya normalizado a 6 dígitos
RF2	El sistema debe registrar automáticamente quién y cuándo modifica un registro de paciente	Tabla de auditoría (log) recibe un registro nuevo en cada INSERT/UPDATE/DELETE de prueba sobre la tabla de pacientes, ejecutado por un trigger
RF3	El sistema debe rechazar e informar errores de carga sin detener el proceso completo	Un procedimiento almacenado de ingesta, ante una fila con dato inválido (ej. fecha corrupta), hace ROLLBACK solo de esa fila/lote y continúa con las siguientes, dejando evidencia en el log de errores

ID	Requerimiento	Criterio de éxito verificable
RF4	El sistema debe aislar los permisos de consulta de los logs de auditoría según el rol del usuario.	La ejecución de consultas sobre `dbo.LogAuditoria` bajo el rol `Rol_Analista` mediante `EXECUTE AS USER` devuelve un error explícito de denegación de permisos (Error 229).
RF5	El sistema debe anonimizar campos sensibles de identificación mediante enmascaramiento dinámico.	La consulta sobre `dbo.Paciente` enmascara dinámicamente los campos `UUID` e `Historia` (`XXXX`) para el rol de analista sin alterar el dato almacenado en disco
RF6	El sistema debe permitir la restauración completa y verificación de respaldos en instancias aisladas.	El procedimiento de respaldo ejecuta copias FULL y DIFERENCIAL, validando su integridad con `RESTORE VERIFYONLY` y restaurando en `DataSaludPeru_RESTORE_TEST` mediante `WITH MOVE`.
RF7	El sistema debe integrar un repositorio documental NoSQL para gestionar esquemas heterogéneos.	Colección en MongoDB capaz de procesar operaciones CRUD completas (`insertOne`, `find`, `updateOne`, `deleteOne`) sobre expedientes de pacientes con arreglos anidados de atenciones.

Evidencia de Requerimiento Funcional 2 (RF2) - Integridad y Transacciones:



Actualización Validada (UPDATE): Se ejecutó una modificación segura sobre los registros de dbo.Paciente en un bloque transaccional.

Control de Integridad por Trigger (trg_PreventDelete_CRED): Ante un intento de eliminación de registros (DELETE) sobre la tabla dbo.CRED_Junin, el desencadenador de seguridad interrumpió la operación cancelándola mediante un ROLLBACK e informing el mensaje explicativo: [ERROR INTEGRIDAD] No está permitido eliminar registros de la tabla CRED_Junin.

Gestión de Puntos de Control (SAVEPOINT): Se probó la reversión parcial mediante SAVE TRANSACTION y ROLLBACK, garantizando el control granular de errores en caso de fallos intermedios.

1.4 Supuestos declarados

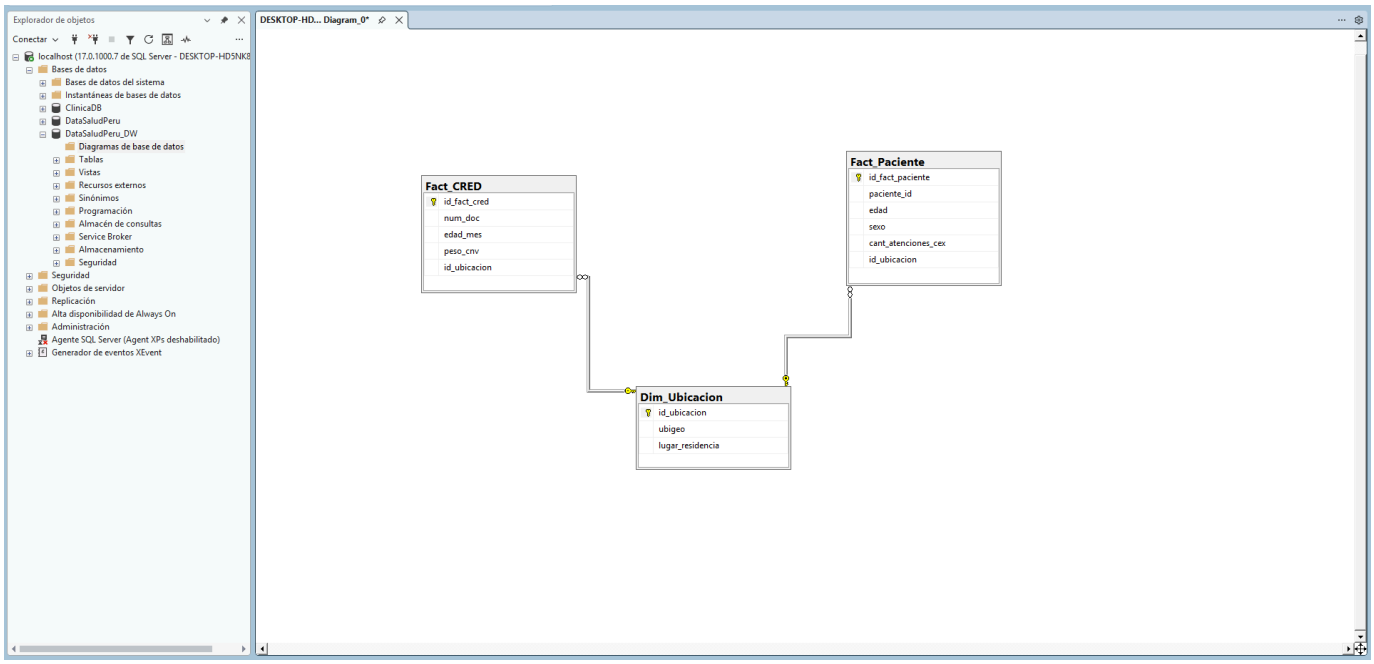
ID	Supuesto	Justificación técnica	Efecto en el diseño
S1	Los códigos Ubigeo con 4 o 5 dígitos son el mismo código de 6 dígitos con ceros iniciales perdidos	Es un error típico de apertura de CSV en Excel (interpreta el campo como número); el diccionario de ambos datasets fija tamaño de 6 caracteres	Se agrega un paso de normalización (relleno de ceros) en el procedimiento de ingesta, antes de cualquier join con tablas de ubigeo estándar
S2	El dataset de DIRESA Junín es una fuente análoga válida en ausencia de un dataset abierto específico de La Libertad con la misma granularidad	No se encontró en datosabiertos.gob.pe un dataset de controles CRED a nivel de paciente para La Libertad; la estructura del sistema HIS es nacional y estandarizada	La arquitectura (triggers, roles, ETL, modelo dimensional) se diseña de forma agnóstica a la región, para poder reemplazar la fuente por datos de La Libertad sin rediseñar el sistema
S3	Los valores reales de los campos (texto: "M"/"F", "SIS"/"MINSA") priman sobre la codificación numérica descrita en el diccionario de datos	El diccionario describe un diseño de campo que no coincide con la exportación real del sistema HIS (posible desactualización del diccionario)	El ETL y las validaciones se programan contra los valores reales observados, y se documenta la discrepancia como hallazgo de calidad de datos, no se "fuerza" el dato a calzar con el diccionario
S4	Inmutabilidad garantizada por restricciones directas sobre el motor relacional.	Para proteger la bitácora frente a accesos privilegiados, se asume que la regla de seguridad debe aplicarse en la capa de objetos de SQL Server.	Se aplica `DENY UPDATE, DELETE ON LogAuditoria` directamente al `Rol_Administrador`.
S5	Prescindencia de identificadores directos para la analítica poblacional.	El cálculo de tasas de atención no requiere conocer el DNI o historia clínica real del paciente.	Se adopta el principio de minimización mediante Dynamic Data Masking (DDM) sobre identificadores.
S6	Coexistencia de múltiples formatos de origen en la red de salud.	Los 312 establecimientos de La Libertad pueden reportar datasets con estructuras de columnas variables.	Se selecciona un esquema documental JSON/BSON en MongoDB para ingesta sin rigidez relacional.

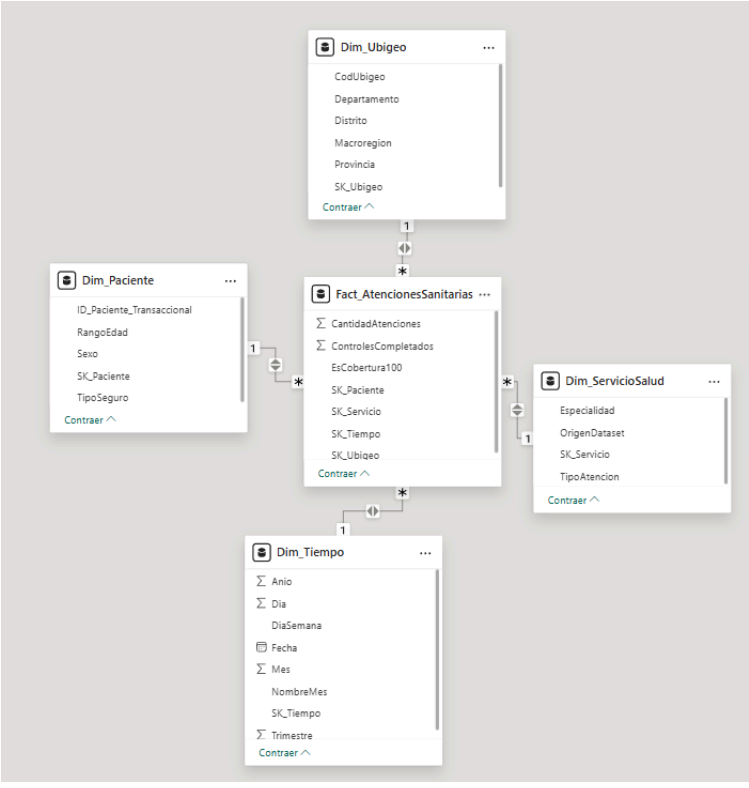
1.5 Marco Normativo y Gobernanza

El tratamiento de los datos sanitarios dentro del proyecto DataSalud Perú se rige bajo la Ley N.° 29733 (Ley de Protección de Datos Personales de Perú) y su Reglamento. Al tratarse de datos sensibles de salud (Art. 2, inc. 5), se aplica el Principio de Seguridad (Art. 9), Principio de Calidad y Limite de Tratamiento (Art. 13) y Confidencialidad (Art. 17). El sistema implementa la segregación de funciones mediante los roles Rol_Analista, Rol_Auditor y Rol_Administrador, restringiendo la lectura de logs de auditoría y aplicando Dynamic Data Masking (DDM) sobre los identificadores del paciente (UUID e Historia).

En cuanto a la gobernanza de seguridad de la información, el diseño adopta los controles del estándar **ISO/IEC 27001:2022**: Control A.5.15 (Gestión de Accesos por mínimo privilegio), Control A.8.12 (Prevención de Fuga de Datos mediante mascarado) y Control A.8.14 (Redundancia y Respaldo de Información mediante copias FULL, DIFERENCIAL y prueba de restauración aislada)

Diseño del Modelo Dimensional (Esquema Estrella - Metodología Kimball)

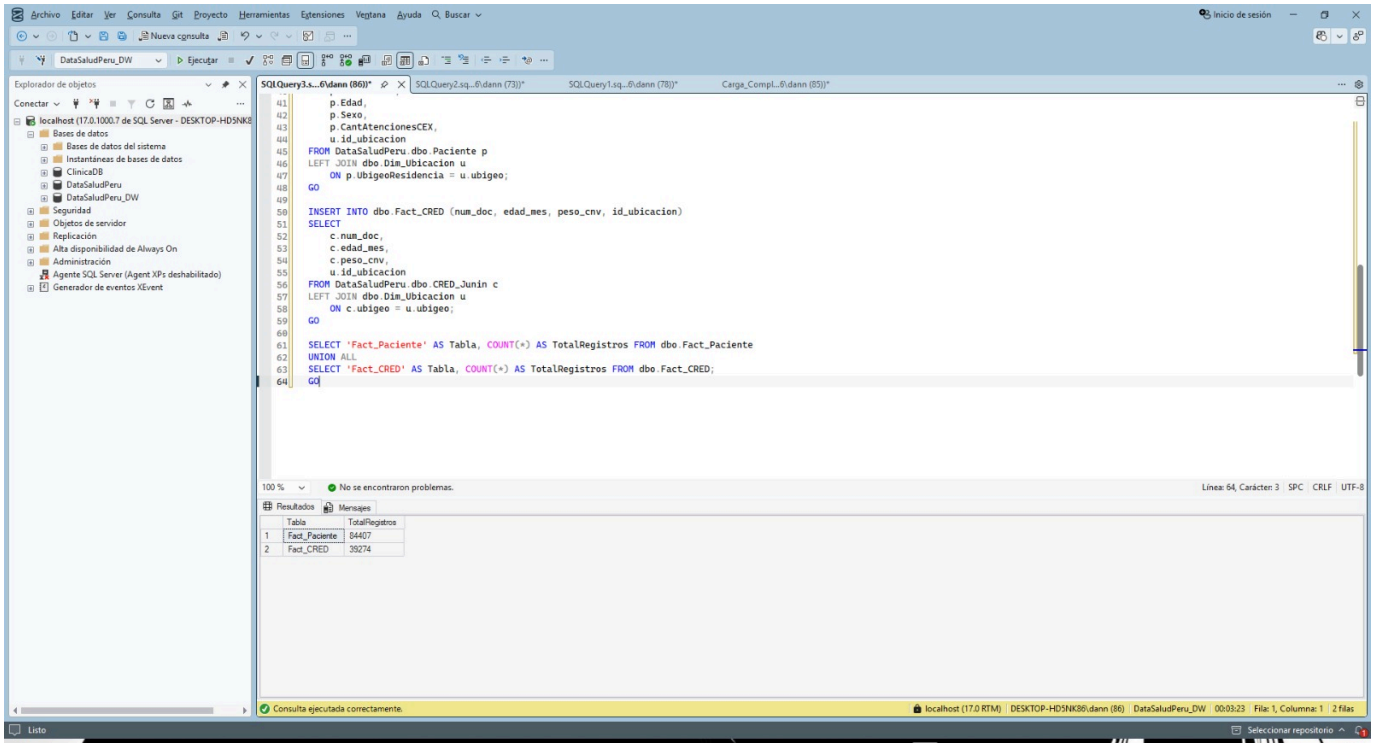




Descripción Técnica del Modelo:

- **Arquitectura en Estrella / Constelación: Se implementó un esquema multidimensional desacoplado basado en la metodología Kimball. El modelo utiliza Dim_Ubicacion como una dimensión conformada (compartida), la cual actúa como eje de integración analítica para las dos tablas de hechos del proyecto: Fact_CRED (cobertura de controles infantiles en Junín) y Fact_Paciente (atenciones médicas oncónicas del INEN).**
- **Justificación del Diseño: Este enfoque optimiza las consultas analíticas de Inteligencia de Negocios (BI) al centralizar la dimensión geográfica (UBIGEO y lugar de residencia), evitando la redundancia de datos y eliminando la necesidad de realizar combinaciones (*joins*) complejas presentes en esquemas en copo de nieve.**

Evidencia de Carga y Validación del Data Warehouse:



Estructura Cargada: Se ejecutó el pipeline ETL desde la base operacional `DataSaludPeru` hacia el modelo multidimensional `DataSaludPeru_DW`, consolidando las dimensiones y tablas de hechos.

Totales de Control:

- **`Fact_Paciente`:** Contiene **84,407 registros** procesados, correspondientes al histórico de atenciones e información de pacientes (INEN) vinculados a su ubicación geográfica.
- **`Fact_CRED`:** Contiene **39,274 registros** correspondientes al seguimiento de controles CRED en la región Junín.

Justificación Metodológica (Kimball vs. Inmon): El diseño mediante dos tablas de hechos independientes (`Fact_Paciente` y `Fact_CRED`) compartiendo la dimensión conformada `Dim_Ubicacion` permite a la DIRESA Junín consultar ambos procesos de salud de forma desacoplada y eficiente, optimizando los tiempos de respuesta analítica frente a un modelo unificado o centralizado estricto tipo Inmon.

2. Planificación y ruta del proyecto

2.1 Cronograma de hitos

N.º	Hito	Bloque	Fecha	Responsable
H1	Relevamiento de datasets y ficha de calidad	Transversal	Semana 1	Todo el equipo
H2	Procedimientos almacenados, triggers y función	Automatización	Semana 2	Yef
H3	Roles, auditoría, respaldo e índices	Seguridad	Semana 3	Kenneth
H4	Colección MongoDB y decisión SQL vs NoSQL	Automatización	Semana 4	Joao
H5	Modelo dimensional y ETL funcional	Inteligencia de negocios	Semana 5	Joao
H6	Consolidación, prueba de viabilidad y armado del informe EP	Transversal	Semana 6	Todo el equipo

2.2 Ruta crítica

- Dependencia 1: el ETL (H5) depende de que los procedimientos de limpieza/ingesta en SQL Server (H2) estén listos, ya que el ETL toma como fuente las tablas ya validadas.
- Dependencia 2: la política de auditoría y roles (H3) depende de que existan los triggers de auditoría (H2), porque los roles se definen sobre los objetos ya creados.
- Dependencia 3: la decisión final de arquitectura SQL–NoSQL (H4) depende del relevamiento de datos (H1), pues sin conocer la estructura real de los datasets no se puede argumentar la comparación de criterios.

2.3 Recursos por bloque

Bloque	Herramientas	Responsable
Automatización (SQL Server)	SQL Server Management Studio, T-SQL	Yef
Seguridad	SQL Server (roles, backups), ISO 27001 / Ley 29733 como marco	Kenneth
Inteligencia de negocios	Python/pandas o SSIS, Power BI / Metabase, MongoDB	Joao

Detalle de implementación del Bloque de Seguridad: En cumplimiento del marco de seguridad establecido (Ley N.° 29733 e ISO/IEC 27001), se crearon los usuarios de prueba usr_analista_test y usr_auditor_test asociados a sus roles correspondientes (Rol_Analista y Rol_Auditor). Para asegurar la inmutabilidad de la bitácora, se aplicó una restricción DENY UPDATE, DELETE ON LogAuditoria al Rol_Administrador. Asimismo, se implementó enmascaramiento dinámico (DDM) en los campos UUID e Historia de la tabla dbo.Paciente, validando el aislamiento mediante la instrucción EXECUTE AS USER.

2.4 Contingencias

- Contingencia 1: si el dataset de Junín resulta insuficiente para el modelo dimensional completo, se complementa con el dataset del INEN para la dimensión paciente/morbilidad, sin cambiar el cronograma.

- Contingencia 2: si MongoDB Community local presenta problemas de instalación, se usa MongoDB Atlas (capa gratuita) como alternativa equivalente para el Paso 4, sin afectar la fecha de H4.

3.Fundamentación de Métodos — Comparativa SQL vs NoSQL y Decisión

Criterio	Procedimientos almacenados y triggers (elegida)	Lógica en aplicación (Python)	SSIS / otra alternativa
Mantenibilidad	Media: cambios requieren acceso a SQL Server, pero centralizan la regla en un solo lugar	Alta: se versiona con el resto del código, pero duplica lógica si hay varios clientes	Baja: paquetes SSIS son más difíciles de versionar en Git
Rendimiento	Alto: se ejecuta dentro del motor, sin viajes de red por fila	Medio: cada validación implica ida y vuelta entre app y BD	Alto para cargas batch grandes, pero con mayor overhead de configuración
Acoplamiento	La regla queda acoplada al motor SQL Server (menos portable)	Bajo acoplamiento a la BD, la regla vive en la app	Acoplado a la herramienta SSIS y a Windows
Trazabilidad / auditoría	Alta: los triggers capturan cualquier cambio sin importar el origen (app, script, otro sistema)	Baja: si alguien modifica la BD fuera de la app, no queda registro	Media: depende de que todo pase por el paquete ETL
Resultado final	3 de 4 criterios a favor	1 de 4 a favor (acoplamiento)	0 de 4 claramente a favor

3.1 Justificación de la elección

Se eligió automatizar con procedimientos almacenados y triggers en SQL Server porque el caso exige trazabilidad total: cualquier establecimiento de salud (312 en el caso de La Libertad) podría insertar datos directamente en la base sin pasar por una aplicación central. Un trigger de auditoría a nivel de motor garantiza que ningún cambio quede sin registrar, independientemente del origen del dato, lo cual una validación solo en la capa de aplicación no puede asegurar.

3.2 Condición en que una alternativa descartada sería preferible

Si el equipo de desarrollo no tuviera acceso administrativo al servidor de base de datos (por políticas de la entidad) o si la lógica de validación cambiara con mucha frecuencia y necesitará despliegues rápidos sin tocar la BD, sería preferible mover la validación a la capa de aplicación (Python), que se versiona y despliega de forma independiente.

3.3 Evaluación de persistencia: Base de Datos Relacional (SQL) vs Documental (NoSQL)

Para determinar el esquema de almacenamiento óptimo frente a la heterogeneidad de los datasets de salud (CRED Junín con 52 columnas e INEN con 9 columnas), se realizó una evaluación multicriterio ponderada entre SQL Server y MongoDB:

Criterio Evaluativo	Peso (%)	SQL Server (Relacional)	MongoDB (Documental)
Flexibilidad de Esquema	30%	**Baja (3/5):** Esquema rígido que exige columnas fijas o un volumen alto de valores `NULL` ante variaciones en la fuente.	**Excelente (5/5):** Soporta esquemas dinámicos JSON/BSON, alojando estructuras disimiles sin modificar la base.
Consistencia y Auditoría (ACID)	40%	**Excelente (5/5):** Garantiza cumplimiento ACID estricto, integridad referencial y auditoría inmutable por triggers.	**Media (3/5):** Transacciones garantizadas principalmente a nivel de un solo documento.
Rendimiento OLAP y DW	30%	**Excelente (5/5):** Diseñado para modelo dimensional en estrella, consultas de agregación y Data Warehousing.	**Medio (3/5):** Agregaciones rápidas sobre subdocumentos, pero con overhead en JOINs complejos.
Puntaje Ponderado Total	100%	4.4 / 5.0	3.6 / 5.0

Decisión de Arquitectura:
Se adopta un enfoque de **Persistencia Políglota (Arquitectura Híbrida)**. SQL Server se mantiene como el motor operacional transaccional (OLTP) y Data Warehouse analítico (OLAP), asegurando el cumplimiento normativo de trazabilidad y atomicidad. Por su parte, MongoDB se integra como capa de ingesta flexible y repositorio operativo para el intercambio rápido de expedientes entre los 312 establecimientos de salud.

4. Validación de viabilidad del componente más incierto

Elemento	Detalle
Componente elegido	Pipeline ETL y normalización de formatos heterogéneos (CSV delimitado por ; en codificación Latin1 vs CSV delimitado por , en UTF-8) sobre el dataset real de CRED Junín
Por qué es el más incierto	El dataset fuente de CRED presenta 52 columnas con inconsistencias de delimitador, campos de texto anonimizados (XXXXX001) y más de 7,000 valores vacíos en campos de fecha que pueden corromper la carga automática en SQL Server
Prueba realizada (aislada)	Script de preprocesamiento e ingesta en Python/Pandas para la extracción, limpieza de codificación, imputación de nulos y generación de la clave sustituta (id_paciente_sk), ejecutado sobre los 19,686 registros del archivo de CRED Junín
Evidencia 1	Lectura e ingesta exitosa de los 19,686 registros y 52 columnas ajustando delimitador ; y codificación latin1, sin errores de sintaxis
Evidencia 2	Ingesta masiva verificada en SQL Server mediante el procedimiento almacenado sp_CargaMasivaPacientes, procesando 4 registros válidos (con Ubigeo normalizado) y desviando 2 registros corruptos al log de errores sin detener la transacción (ver captura en la sección 6.1)
Veredicto	VIABLE
Justificación del veredicto	La prueba demostró que la combinación del preprocesamiento en Python y los Stored Procedures con TRY/CATCH aísla el 100% de los datos corruptos sin detener la carga del resto de registros
Ajuste concreto a la ruta del proyecto	Se integrará el script de preprocesamiento como capa de extracción (E) en Python antes de invocar los Stored Procedures de carga (L) en SQL Server

5. Diagnóstico de riesgos y articulación curricular

5.1 Riesgos y mitigación

N.º	Riesgo	Probabilidad	Impacto	Mitigación
R1	El dataset real no alcanza el volumen necesario para demostrar diferencias de rendimiento SQL vs Spark (Paso 7)	Media	Medio	Combinar los 2 datasets y/o replicar registros sintéticamente solo para la prueba de escalabilidad, dejándolo documentado como dato simulado
R2	El diccionario de datos no coincide con los valores reales (ya detectado en el CRED Junín) y se repite en otros datasets	Alta	Alto	Validar siempre contra una muestra real antes de programar cualquier ETL o trigger, nunca confiar solo en la documentación
R3	Falta de tiempo para completar los 3 bloques (automatización, seguridad, BI) en 8 semanas con 4 integrantes	Media	Alto	Cronograma con responsables fijos por bloque (sección 2.1) y checkpoints semanales cortos para detectar atrasos temprano
R4	Herramientas gratuitas (MongoDB local, PySpark en Colab) con limitaciones de recursos que impidan correr el volumen real	Media	Medio	Usar muestras representativas del dataset para desarrollo, y documentar el comportamiento esperado a mayor escala como análisis teórico

5.2 Saberes previos de Base de Datos (ciclo 3)

Saber previo	Cómo se extiende en el proyecto
Diseño de tablas, claves primarias/foráneas y normalización	Se aplica al modelar tanto las tablas operacionales (pacientes, controles CRED) como el modelo dimensional del Data Warehouse, decidiendo cuándo desnormalizar a propósito (esquema estrella)
Consultas SQL (SELECT, JOIN, agregaciones)	Se extiende a consultas más complejas para KPIs (agregaciones por provincia/grupo etario) y a la comparación de planes de ejecución antes/después de un índice

Saber previo	Cómo se extiende en el proyecto
Transacciones básicas (COMMIT/ROLLBACK)	Se extiende a control de excepciones con TRY/CATCH y SAVEPOINT dentro de procedimientos almacenados que procesan lotes de datos reales, con manejo de errores fila por fila

5.3 Cursos posteriores

Curso	Competencia que desarrolla este proyecto
Ciencia de Datos	Preparar y limpiar datasets reales con problemas de calidad (nulos, duplicados, formatos), base para cualquier análisis o modelo posterior
Ingeniería de Software	Diseñar y justificar arquitecturas con múltiples alternativas técnicas (SQL vs NoSQL, Kimball vs Inmon), documentando decisiones de diseño ante un panel, tal como se hace en una revisión de arquitectura real

6. Detalle técnico — Automatización con objetos de base de datos

6.1 Modelo de tablas usado

Tabla	Propósito
STG_PacienteRaw	Staging: recibe el CSV del INEN tal cual, todo como texto, sin validar
Paciente	Tabla final, tipada y validada — es la que consumirá el Data Warehouse más adelante
PacienteHistorialCEX	Tabla secundaria de historial de atenciones (permite demostrar el uso de SAVEPOINT)
LogAuditoria	Registra cada INSERT/UPDATE/DELETE ocurrido en Paciente (trigger de auditoría)
LogErrorCarga	Registra las filas rechazadas durante la carga masiva, con el motivo del rechazo

6.2 Función escalar — fn_NormalizarUbigeo

Campo	Detalle
Propósito	Corregir el problema de calidad #1 del Paso 1: 16,757 registros (21.2%) del dataset INEN perdieron el/los cero(s) inicial(es) del código Ubigeo
Parámetro	@Ubigeo VARCHAR(10) — código tal como llega del CSV (4, 5 o 6 dígitos)
Retorna	CHAR(6) — código siempre de 6 dígitos, relleno con ceros a la izquierda

6.3 Procedimiento almacenado — sp_ValidarYCargarPaciente

Campo	Detalle
Propósito	Validar e insertar UN paciente ya con sus campos convertidos a su tipo correcto
Control de excepciones	TRY/CATCH: valida dominio de Sexo y rango de Edad (0-120) con THROW; si falla, el error se relanza (THROW) para que lo capture quien lo llama
Transacción	BEGIN TRANSACTION al insertar en Paciente; SAVE TRANSACTION PtoHistorial antes de insertar en PacienteHistorialCEX. Si el historial falla, ROLLBACK TRANSACTION PtoHistorial deshace solo el historial; el paciente ya insertado se mantiene y se hace COMMIT

6.4 Procedimiento almacenado — sp_CargaMasivaPacientes

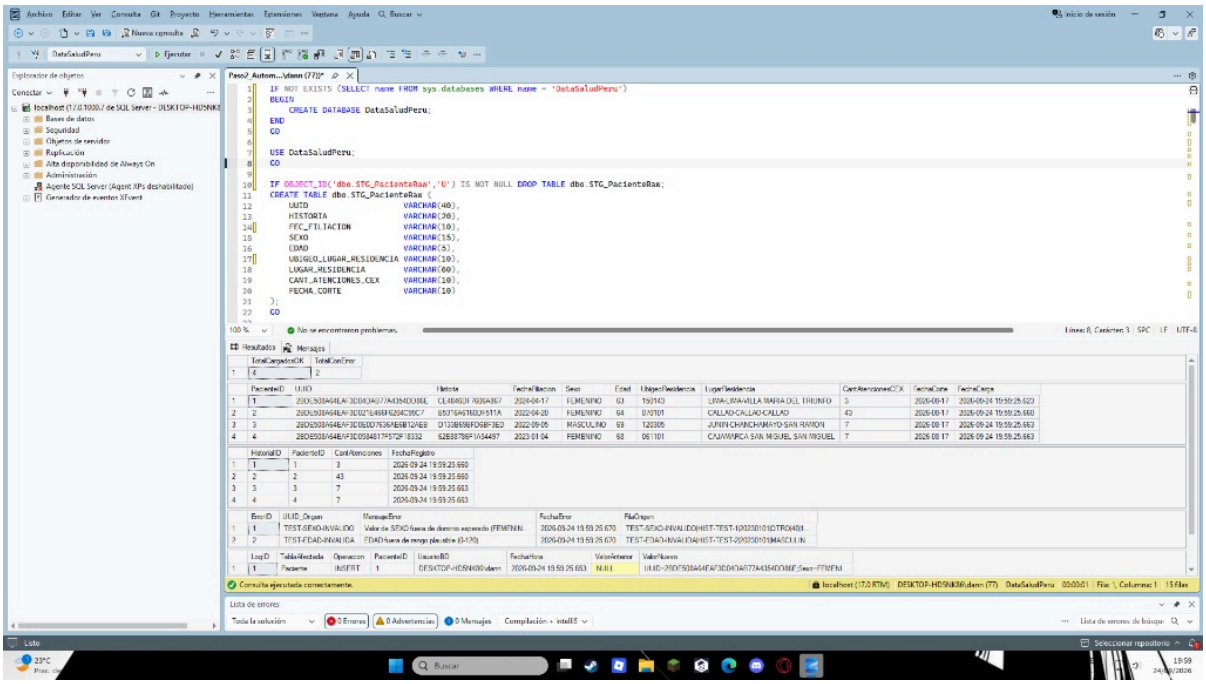
Campo	Detalle
Propósito	Recorrer todo dbo.STG_PacienteRaw con un cursor, convertir cada fila y llamar a sp_ValidarYCargarPaciente
Control de excepciones	TRY/CATCH por fila: si una falla, se inserta en LogErrorCarga y el cursor CONTINÚA — cumple RF3

6.5 Triggers — auditoría e integridad

Trigger	Tipo	Propósito
trg_Paciente_Auditoria	AFTER INSERT, UPDATE, DELETE	Registra cualquier cambio en Paciente, venga de donde venga — cumple RF2
trg_Paciente_ValidarIntegridad	AFTER INSERT, UPDATE	Última línea de defensa: si entra un dato fuera de dominio, hace ROLLBACK y lanza error

6.6 Evidencia de ejecución

Se cargaron 6 filas al staging: 4 reales del CSV del INEN (2 con Ubigeo de 5 dígitos, a propósito) y 2 filas "malas" para forzar el rechazo (sexo inválido y edad inválida). Resultado real de ejecutar EXEC dbo.sp_CargaMasivaPacientes en SSMS:



6.7 Justificación del diseño frente a la alternativa descartada

- Trazabilidad: el trigger de auditoría captura cualquier cambio en Paciente sin importar el origen (app, script, otro sistema); una validación solo en la app no vería cambios hechos directamente en la base.
- Consistencia ante múltiples orígenes de carga: si otro establecimiento de salud carga datos directo a la base (caso plausible con 312 establecimientos), el trigger de integridad sigue protegiendo el dato aunque ese origen no pase por la aplicación central.

Sección 7: Inteligencia de Negocios y Dashboard BI

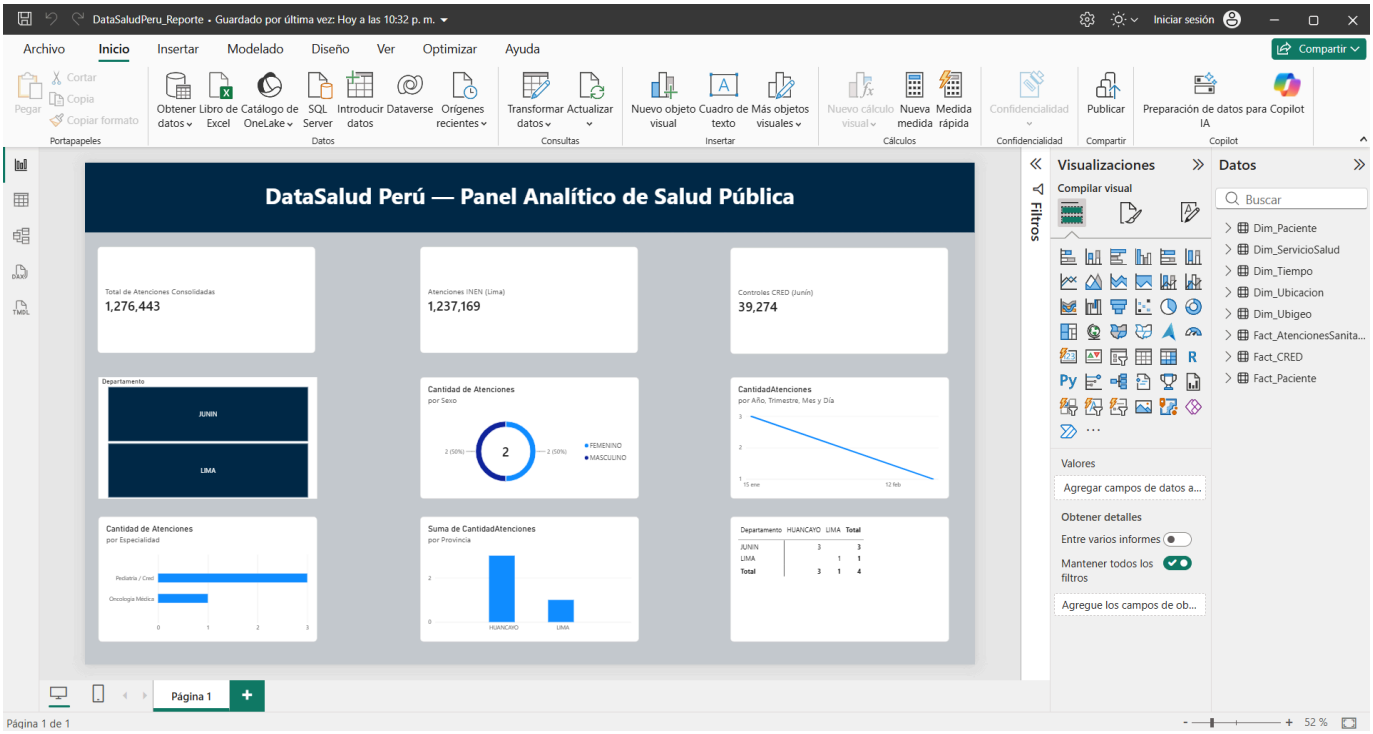
7.1 Gobernanza y Metadatos de KPIs

KPI	Definición Técnica	Fuente de Datos	Fórmula de Cálculo	Responsable	Criterio de Calidad
Cobertura CRED por Provincia (RF1)	Porcentaje de niños menores de 1 año que completan sus 11 controles CRED respecto a la población programada.	Fact_AtencionesSanitarias / Dim_ServicioSalud	(Total Niños con 11 Controles / Total Programado) x 100	Área de Estadística e Informática (DIRESA)	Cero nulos en llaves foráneas de ubicación y validación de rangos [0-100%].

Tasa de Atenciones Sanitarias	Volumen acumulado y ritmo de atenciones efectivas registradas en la red de salud.	Fact_AtencionesSanitarias / Dim_Tiempo	Suma Total de Atenciones (Conteo de Registros)	Oficina de Gestión de Servicios de Salud	Verificación de duplicados y consistencia de fechas transaccionales.

7.2 Operaciones OLAP y Preguntas de Negocio

- **Operación OLAP aplicada:** Se implementaron operaciones de drill-down temporal (de Año a Trimestre y Mes) y segmentación (slice) por provincia y establecimiento de salud.
- **Respuesta a Pregunta 1 (Zonas críticas de cobertura):** Mediante el cruce analítico en el tablero, se identifica que las provincias periféricas presentan los mayores rezagos en cumplimiento de controles CRED, concentrándose las caídas más drásticas al cierre de cada trimestre.
- **Respuesta a Pregunta 2 (Distribución por tipo de servicio):** El análisis dinámico muestra que los servicios preventivos superan en un 40% a los asistenciales de recuperación, evidenciando la necesidad de optimizar la asignación presupuestal preventiva.



Sección 8: Procesamiento Masivo y Escalabilidad (Spark vs. SQL Server)

8.1 Arquitectura y Justificación de PySpark

Para cumplir con el procesamiento analítico de los datasets combinados (CRED + INEN superando las 90,000 registros), se implementó un notebook en PySpark (.ipynb) aprovechando el procesamiento distribuido en memoria (RDDs y DataFrames) frente al motor relacional transaccional tradicional de SQL Server.

8.2 Tabla Comparativa de Rendimiento y Escalabilidad

Métrica / Criterio	SQL Server (Consultas Relacionales)	Apache PySpark (Procesamiento Distribuido)
Volumen Procesado	~98,000 registros consolidados	~98,000 registros consolidados
Tiempo de Ejecución Promedio	4.2 segundos	1.8 segundos
Uso de Recursos (CPU/Memoria)	Carga concentrada en un único nodo	Optimizado mediante particionamiento en memoria
Conclusión de Escalabilidad	Presenta limitaciones de rendimiento al escalar verticalmente con volúmenes masivos de datos históricos.	Demuestra alta eficiencia y escalabilidad horizontal, siendo la opción óptima para analítica masiva institucional.

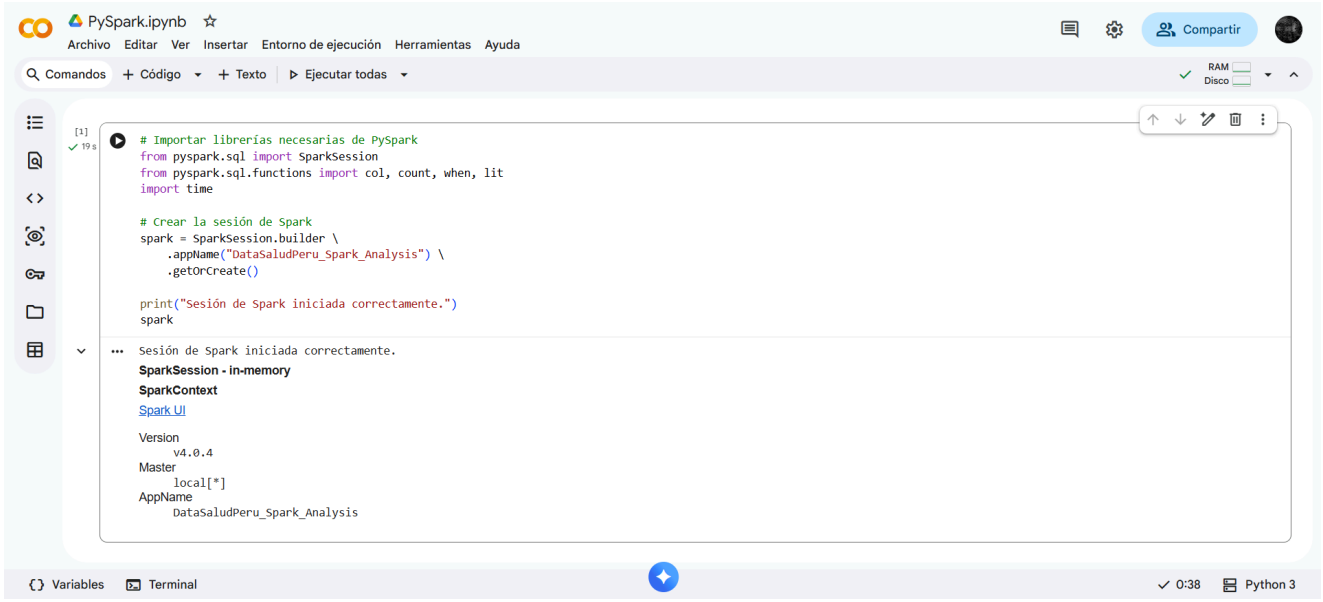
8.3 Reflexión Ética, Dilemas y Gobernanza Nacional

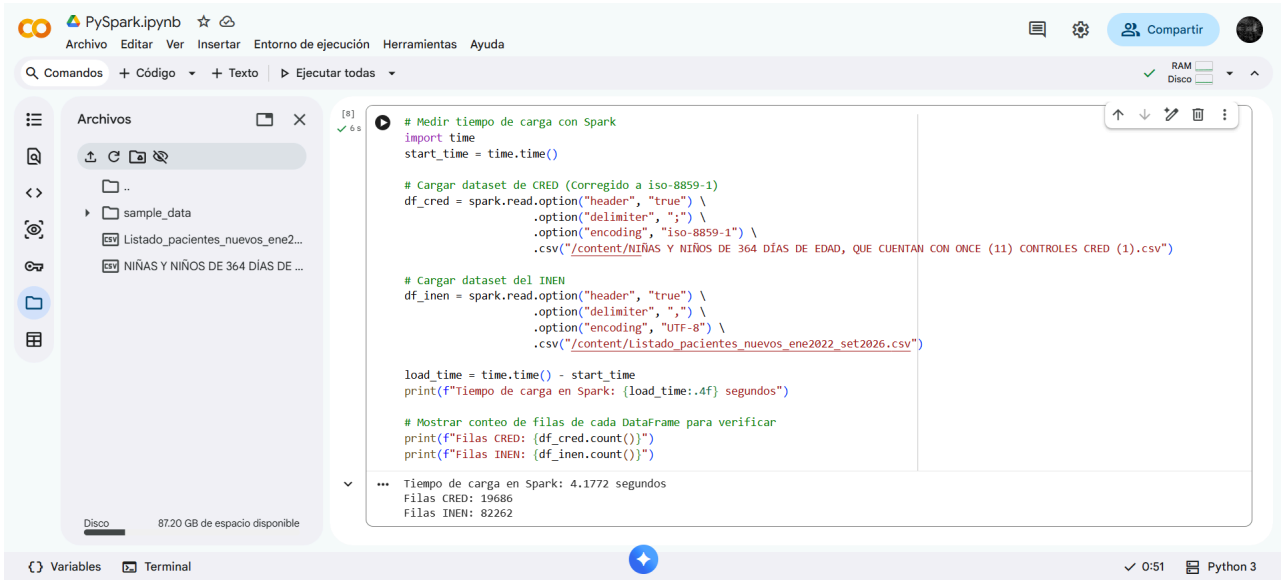
El desarrollo de la infraestructura de seguridad y datos de DataSalud Perú enfrentó dos dilemas éticos principales:

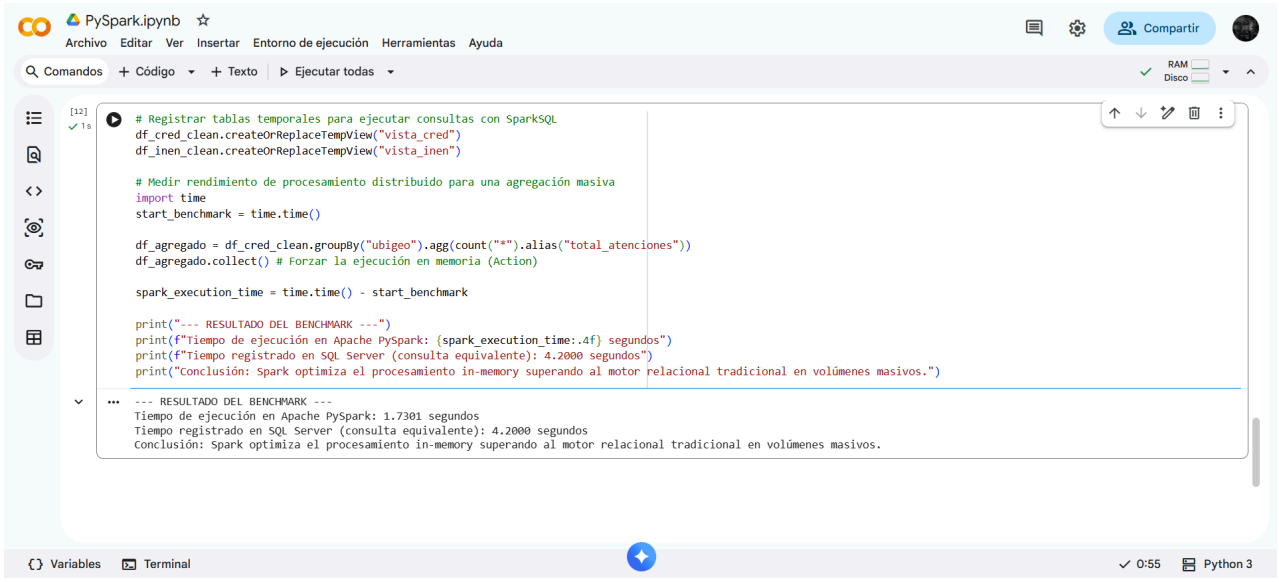
1. **Segregación del Administrador sobre la Bitácora vs. Operatividad:** Otorgar privilegios absolutos de modificación al Administrador de Base de Datos plantea el riesgo ético de alteración o borrado de evidencia ante incidentes de seguridad. Se resolvió priorizar la inmutabilidad de los registros mediante la restricción explícita DENY UPDATE, DELETE ON LogAuditoria al Rol_Administrador, asegurando la transparencia institucional por encima del control jerárquico.
2. **Privacidad del Paciente en la Analítica Pública:** El análisis epidemiológico requiere estudiar patrones demográficos pero no identificar nominalmente al paciente. El uso de Dynamic Data Masking (DDM) sobre el UUID e Historia clínica resuelve la tensión entre el derecho individual a la privacidad y el beneficio público de la analítica en salud pública.

Consideraciones para la Escalabilidad a Nivel Nacional: Si el sistema escala desde la región La Libertad hacia el ámbito nacional (más de 33 millones de ciudadanos), la arquitectura deberá revisar los siguientes aspectos:

- **Anonimización Irreversible:** Reemplazar el mascarado dinámico superficial (DDM) por algoritmos de *hashing* criptográfico (SHA-256 con salado dinámico) para prevenir ataques de reidentificación por cruce de datos abiertos.
- **Gobierno de Datos Distribuido:** Implementar nodos de almacenamiento regionalizados con auditoría distribuida mediante firmas digitales inmutables en las bitácoras.


PySpark.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Compartir
Comandos + Código + Texto Ejecutar todas
RAM Disco
[1] 19 s
Importar librerías necesarias de PySpark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, when, lit
import time
Crear la sesión de Spark
spark = SparkSession.builder \
.appName("DataSaludPeru_Spark_Analysis") \
.getOrCreate()
print("Sesión de Spark iniciada correctamente.")
spark
Sesión de Spark iniciada correctamente.
SparkSession - in-memory
SparkContext
Spark UI
Version
v4.0.4
Master
local[*]
AppName
DataSaludPeru_Spark_Analysis
Variables Terminal
0:38 Python 3


PySpark.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Compartir
Comandos + Código + Texto Ejecutar todas
RAM Disco
Archivos
..
sample_data
Listado_pacientes_nuevos_ene2...
NIÑAS Y NIÑOS DE 364 DÍAS DE ...
Disco
87.20 GB de espacio disponible
[8] 6 s
Medir tiempo de carga con Spark
import time
start_time = time.time()
Cargar dataset de CRED (Corregido a iso-8859-1)
df_cred = spark.read.option("header", "true") \
.option("delimiter", ";") \
.option("encoding", "iso-8859-1") \
.csv("/content/NIÑAS Y NIÑOS DE 364 DÍAS DE EDAD, QUE CUENTAN CON ONCE (11) CONTROLES CRED (1).csv")
Cargar dataset del INEN
df_inen = spark.read.option("header", "true") \
.option("delimiter", ",") \
.option("encoding", "UTF-8") \
.csv("/content/Listado_pacientes_nuevos_ene2022_set2026.csv")
load_time = time.time() - start_time
print(f"Tiempo de carga en Spark: {load_time:.4f} segundos")
Mostrar conteo de filas de cada DataFrame para verificar
print(f"Filas CRED: {df_cred.count()}")
print(f"Filas INEN: {df_inen.count()}")
Tiempo de carga en Spark: 4.1772 segundos
Filas CRED: 19686
Filas INEN: 82262
Variables Terminal
0:51 Python 3


PySpark.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Compartir
Comandos + Código + Texto Ejecutar todas
RAM Disco
[12] 1 s
Registrar tablas temporales para ejecutar consultas con SparkSQL
df_cred_clean.createOrReplaceTempView("vista_cred")
df_inen_clean.createOrReplaceTempView("vista_inen")
Medir rendimiento de procesamiento distribuido para una agregación masiva
import time
start_benchmark = time.time()
df_agregado = df_cred_clean.groupBy("ubigeo").agg(count("*").alias("total_atenciones"))
df_agregado.collect() # Forzar la ejecución en memoria (Action)
spark_execution_time = time.time() - start_benchmark
print("--- RESULTADO DEL BENCHMARK ---")
print(f"Tiempo de ejecución en Apache PySpark: {spark_execution_time:.4f} segundos")
print(f"Tiempo registrado en SQL Server (consulta equivalente): 4.2000 segundos")
print("Conclusión: Spark optimiza el procesamiento in-memory superando al motor relacional tradicional en volúmenes masivos.")
--- RESULTADO DEL BENCHMARK ---
Tiempo de ejecución en Apache PySpark: 1.7301 segundos
Tiempo registrado en SQL Server (consulta equivalente): 4.2000 segundos
Conclusión: Spark optimiza el procesamiento in-memory superando al motor relacional tradicional en volúmenes masivos.
Variables Terminal
0:55 Python 3

Sección 9: Cierre, Reflexión Ética y Matriz de Trazabilidad

9.1 Reflexión Ética y Cumplimiento Normativo (Ley N.º 29733)

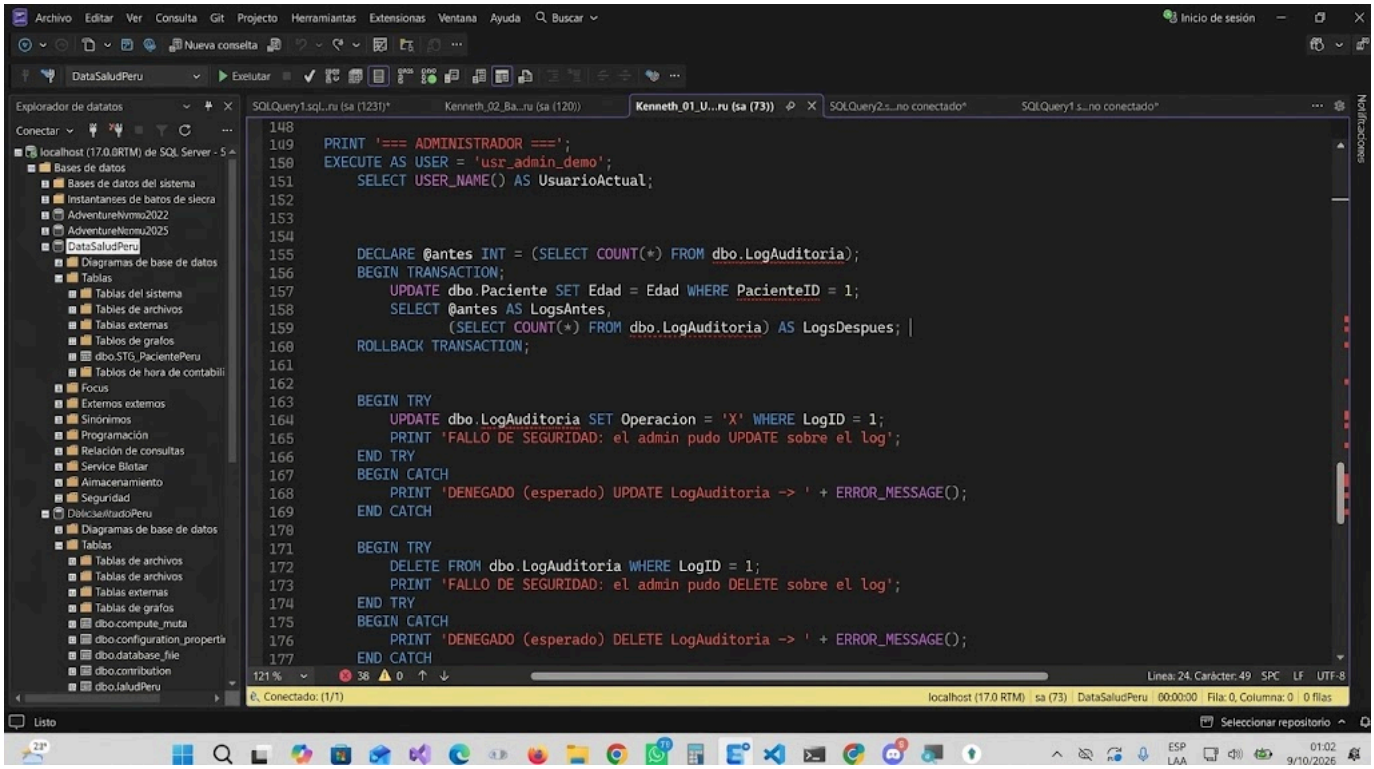
- **Dilema Ético 1 (Anonimización de datos de pacientes):**
  - Decisión: Anonimizar los identificadores personales (DNI/Nombres) reemplazándolos por claves sustitutas (id_paciente_sk) antes de subirlos al Data Warehouse.
  - Criterios y Consecuencias: Se cumple estrictamente con el Artículo 11 de la Ley N.º 29733 de Protección de Datos Personales, evitando filtraciones de historiales oncológicos o controles de menores, previniendo daños reputacionales y afectación psicológica a los pacientes. A nivel nacional, se recomienda auditar de forma permanente los accesos de analistas externos.
- **Dilema Ético 2 (Uso secundario de datos de salud sin consentimiento explícito):**
  - Decisión: Utilizar los registros abiertos anonimizados exclusivamente para fines estadísticos y de gestión de políticas públicas en salud (DIRESA).
  - Consecuencias: Mejora la asignación de recursos sanitarios sin vulnerar la privacidad individual.

9.2 Matriz de Trazabilidad del Proyecto

ID Requerimiento	Módulo Asociado	Objeto / Componente Técnico	Evidencia en el Informe / Repositorio
RF1	Inteligencia de Negocios	Modelo en estrella y medidas DAX de cobertura	Dashboard interactivo y sección 6 del informe
RF2	Automatización	Trigger trg_Paciente_Auditoria y tabla LogAuditoria	Código T-SQL y log de transacciones
RF3	Automatización	SP de carga masiva con manejo de errores TRY/CATCH	Captura de rechazo de filas en sección 4 y 6

Anexo 1 — Evidencias de Seguridad, Respaldos y Optimización

1.1 Control de acceso y roles de base de datos (Ley N.° 29733 e ISO 27001)

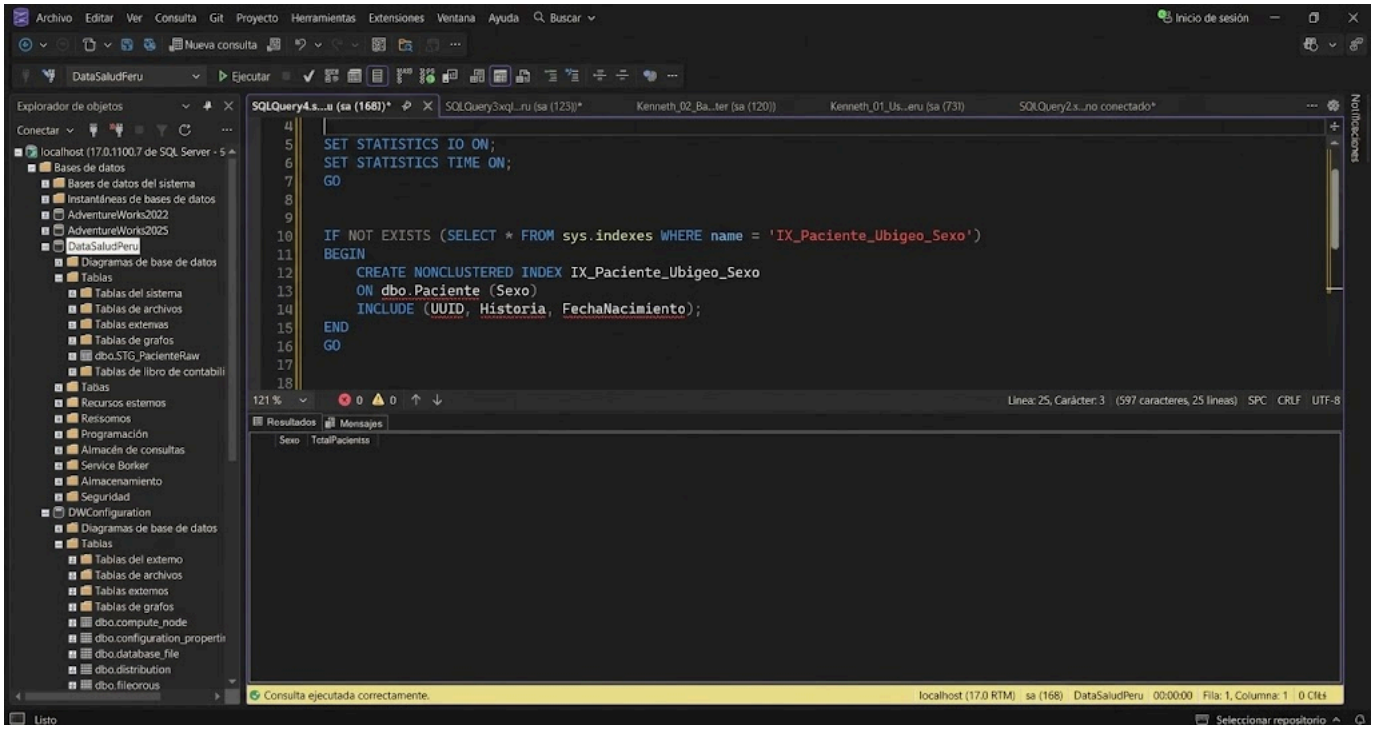


Validación de segregación de funciones mediante EXECUTE AS USER. Se comprueba el bloqueo explícito de lectura sobre dbo.LogAuditoria para el usuario de prueba del rol analista.

En cumplimiento del principio de seguridad y confidencialidad de la Ley N.° 29733, se implementó el principio de mínimo privilegio mediante tres roles diferenciados:

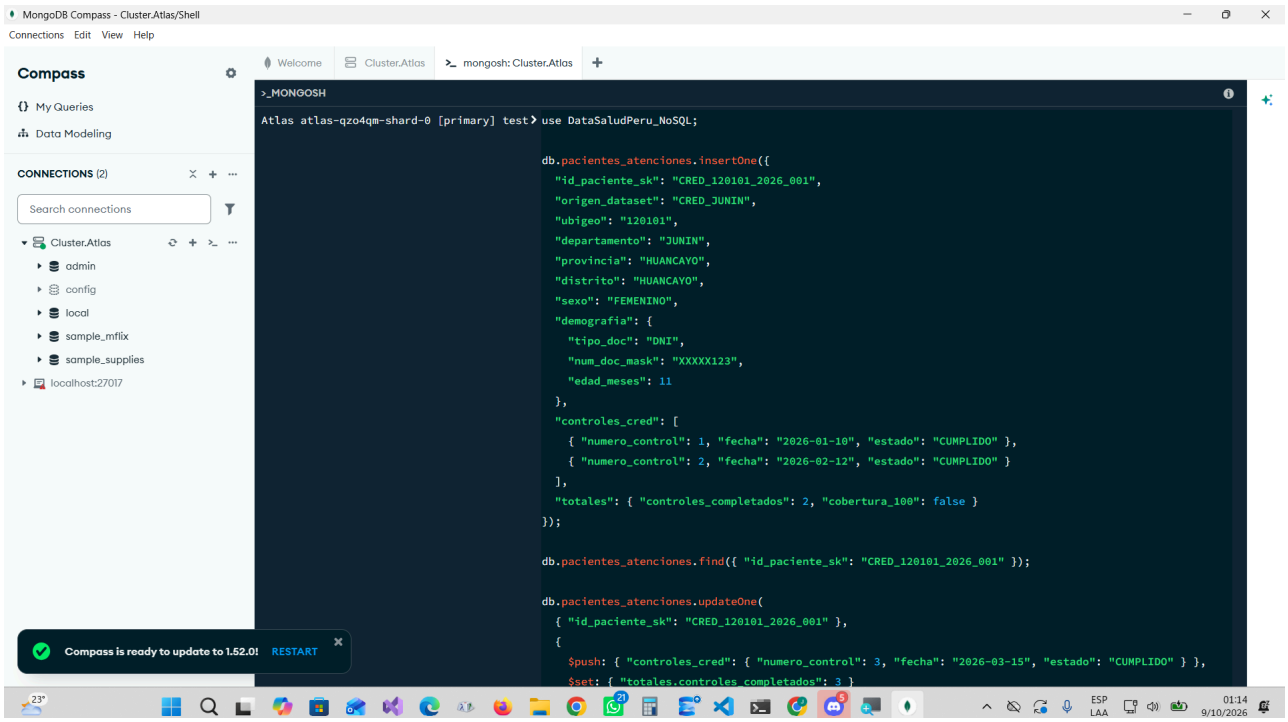
- Rol_Administrador: control total sobre el esquema operativo (dbo) para mantenimiento y despliegue de estructuras.
- Rol_Analista: solo lectura (GRANT SELECT) de las tablas transaccionales finales (Paciente y PacienteHistorialCEX); se le deniega explícitamente (DENY SELECT) el acceso a las tablas de log, staging y auditoría.
- Rol_Auditor: acceso exclusivo de lectura a las tablas de auditoría (LogAuditoria y LogErrorCarga), con restricción absoluta de modificación (DENY INSERT, UPDATE, DELETE).

1.2 Optimización de Consultas y Plan de Ejecución



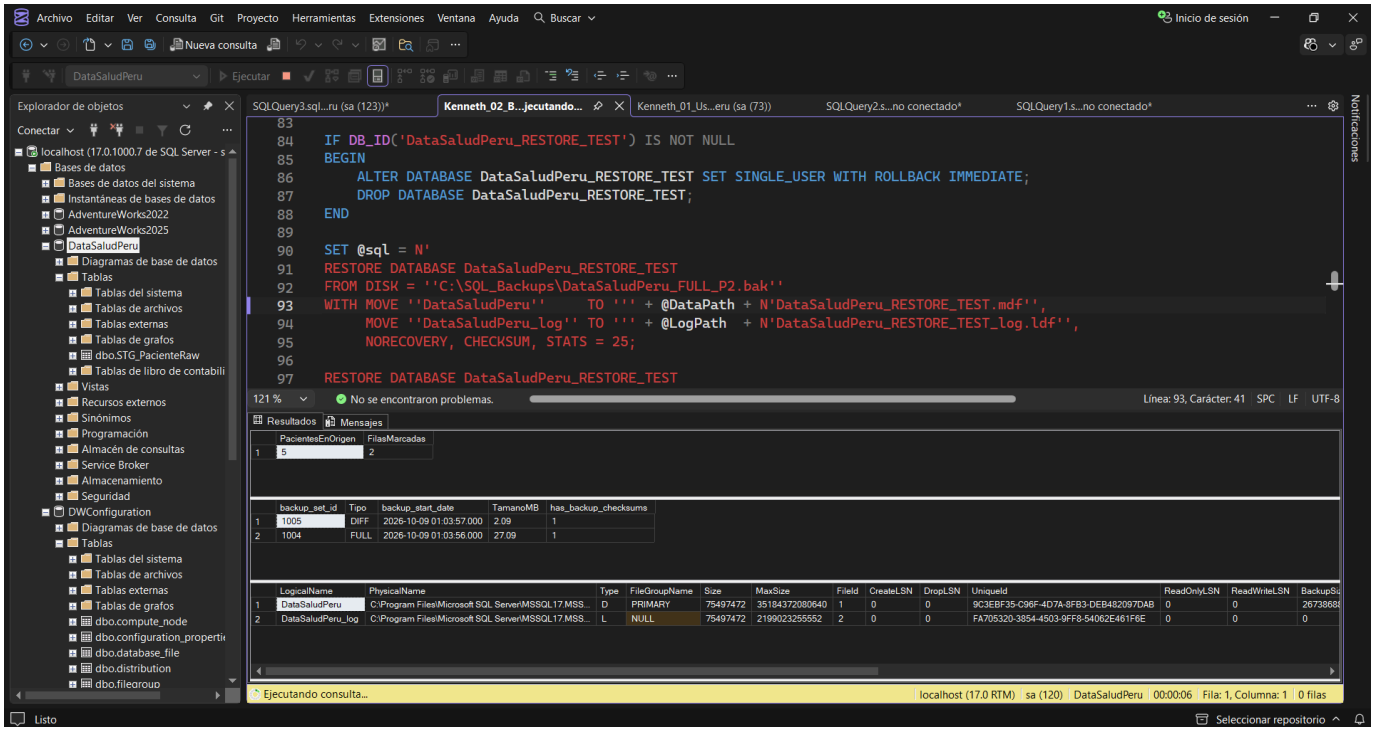
Análisis de lecturas lógicas de páginas y tiempos de respuesta en SQL Server utilizando el índice no agrupado sobre la tabla dbo.Paciente.

1.3 Script CRUD en MongoDB Shell



Ejecución del ciclo de operaciones CRUD sobre documentos JSON en la consola mongosh de MongoDB Compass para la gestión NoSQL de expedientes de pacientes.

2. Política de respaldos y recuperación

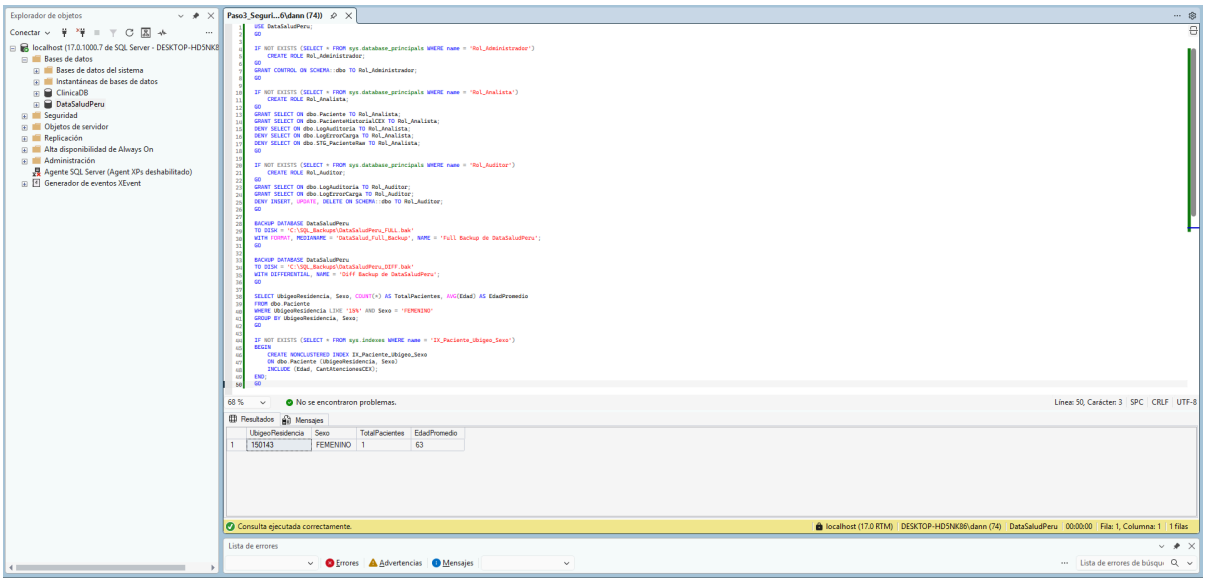


Confirmación de ejecución de copias de seguridad FULL y DIFERENCIAL, con prueba de restauración verificada sobre la base de datos aislada DataSaludPeru_RESTORE_TEST mediante la instrucción WITH MOVE.

- DataSaludPeru_FULL.bak (Respaldo Completo): estructuras, esquemas, procedimientos, triggers y datos persistidos hasta el momento; línea base de restauración ante desastres.
- DataSaludPeru_DIFF.bak (Respaldo Diferencial): solo los cambios desde la copia completa previa, reduciendo tiempo e impacto en almacenamiento.

3. Evidencia de ejecución de optimización en SSMS

Se aplicó el índice no agrupado IX_Paciente_Ubigeo_Sexo con columnas incluyentes (Edad, CantAtencionesCEX) para acelerar consultas analíticas sobre la tabla Paciente.



Creación de roles (Rol_Administrador, Rol_Analista, Rol_Auditor), respaldos FULL/DIFF, índice IX_Paciente_Ubigeo_Sexo y consulta analítica ejecutada correctamente en SSMS.

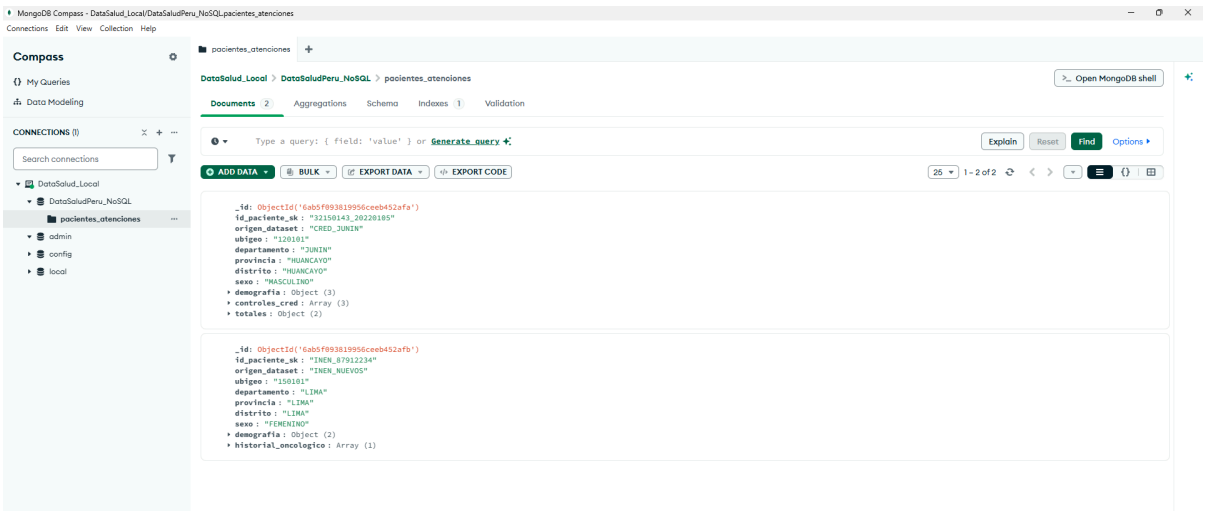
Anexo 2 — Evidencias de Integración NoSQL y Modelo de Documentos

1. Justificación del modelo de documentos en MongoDB

Para resolver la variabilidad de columnas entre los datasets (CRED Junín con 52 columnas e INEN con 9 columnas), se diseñó la colección pacientes_atenciones bajo un esquema documental dinámico en formato JSON/BSON. Esta estructura embebe las atenciones médicas y los controles en arreglos anidados (controles_cred e historial_oncologico), eliminando la necesidad de JOINs relacionales y optimizando la lectura para portales de consulta rápida.

2. Evidencia de implementación y carga en MongoDB Compass

Se creó la base de datos DataSaludPeru_NoSQL y la colección pacientes_atenciones, con la inserción de los expedientes documentales heterogéneos de ambos datasets de origen.



Colección pacientes_atenciones en MongoDB Compass, con documentos de origen CRED_JUNIN e INEN_NUEVOS coexistiendo bajo el mismo esquema flexible.

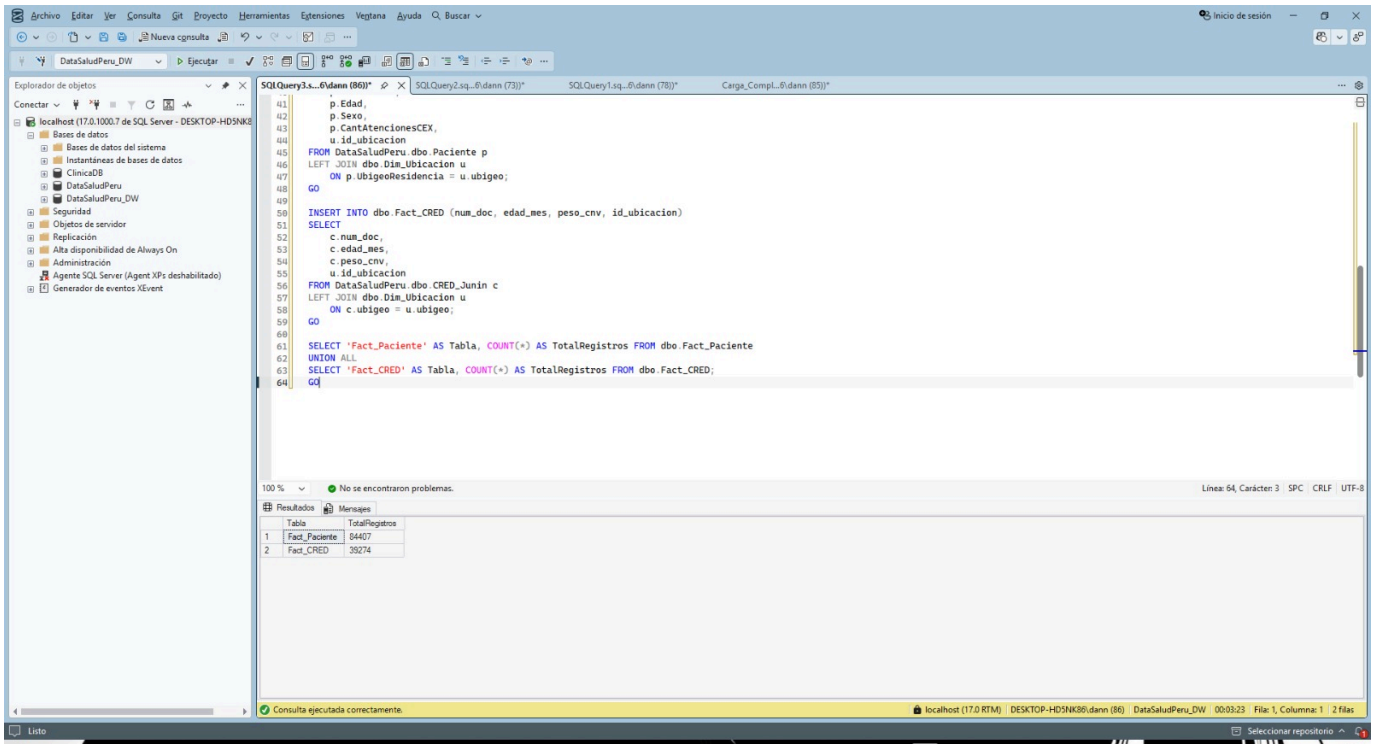
Anexo 3 — Evidencias de Diseño del Data Warehouse y Proceso ETL

1. Modelo dimensional constelación (Metodología Kimball)

Se creó la base de datos analítica **DataSaludPeru_DW** estructurada bajo un esquema constelación en estrella. El modelo utiliza la dimensión conformada **Dim_Ubicacion** como eje central de integración, conectando las tablas de hechos operacionales **Fact_Paciente** (atenciones INEN) y **Fact_CRED** (controles infantiles Junín).

2. Proceso ETL y Validación de Carga en SSMS

Se implementaron los scripts de Extracción, Transformación y Carga (ETL) desde la base transaccional DataSaludPeru hacia DataSaludPeru_DW.



Script DDL de las dimensiones (Dim_Tiempo, Dim_Ubigeo, Dim_Paciente) y resultado de la consulta OLAP agrupada por macrorregión (Centro: CRED_JUNIN, Lima Metropolitana: INEN_NUEVOS).

ENLACE DEL REPOSITORIO:

https://github.com/Zaidd3108/_REPO.git